Ingestão e busca semântica com LangChain e PostgreSQL

Análise do desafio da Full Cycle • Preparado para Gabriel Mendes • 26 de outubro de 2026

Você precisa construir uma aplicação Python que leia um PDF, prepare uma base de consulta no PostgreSQL e responda perguntas no terminal usando somente evidências recuperadas desse documento. O resultado é um pequeno sistema de RAG: recuperação de trechos seguida de geração de resposta.

O resultado esperado

Ao iniciar o chat, o usuário digita uma pergunta. A aplicação busca os trechos pertinentes, entrega esse contexto ao modelo e mostra uma resposta fundamentada. Quando não houver informação suficiente, deve devolver a frase de recusa definida no enunciado.

O que cada tecnologia faz

Componente	Papel no projeto
Python + LangChain	Organizam leitura, divisão do texto, chamadas aos modelos e consulta à base.
Modelo de embeddings	Transforma trechos e perguntas em vetores numéricos comparáveis. Não redige a resposta.
PostgreSQL + pgvector	Persistem os trechos e permitem encontrar vetores próximos ao vetor da pergunta.
Modelo de linguagem (LLM)	Recebe a pergunta e os trechos recuperados; redige a resposta dentro das regras.
Docker Compose	Disponibiliza o banco local de forma reproduzível.

A principal dificuldade

O desafio não termina quando a conexão com o banco funciona. É preciso preservar a informação durante a extração, recuperar o trecho certo e impedir que a resposta complete lacunas com conhecimento externo. Cada uma dessas etapas precisa de uma verificação própria.

Como usar este documento

As páginas seguintes separam **exigências do enunciado** de **recomendações técnicas**. Você encontrará arquitetura, responsabilidades dos arquivos, cuidados com versões, sequência de implementação e critérios para conferir a entrega.

Escopo desta análise: leitura do enunciado aberto no Chrome, inspeção dos arquivos de configuração do repositório de referência e consulta à documentação oficial. O software não foi implementado ou executado; o conteúdo de document.pdf não foi analisado.

O que é obrigatório

Fonte principal: enunciado da plataforma [1]. A matriz abaixo transforma os requisitos em verificações práticas.

Exigência	O que implementar	Como conferir
Stack definida	Python, LangChain e PostgreSQL com pgvector; banco pelo Docker Compose.	Ambiente sobe e a extensão vector está disponível.
Ingestão do PDF	Ler document.pdf e dividir com chunk_size=1000 e chunk_overlap=150.	Inspecionar tamanhos, conteúdo e configuração do splitter.
Persistência vetorial	Gerar embedding para cada chunk e gravar no banco.	Após reiniciar o chat, os trechos continuam consultáveis.
Busca semântica	Vetorizar a pergunta e consultar k=10.	Recuperar até 10 registros; se houver menos, retornar os disponíveis.
Resposta fundamentada	Concatenar trechos, aplicar o prompt fornecido e chamar a LLM.	Resposta sustentada por texto explícito no contexto.
Recusa	Usar a frase definida para informação ausente ou opinião.	Conferir literalmente a frase de recusa nos testes negativos.
Interface CLI	Ler perguntas e imprimir respostas no terminal.	Executar python src/chat.py e fazer várias perguntas.
Estrutura e entrega	Fazer fork; manter arquivos exigidos; publicar código e README.	Repositório público com instruções reproduzíveis.

Estrutura que deve permanecer reconhecível

```
docker-compose.yml      # Banco e bootstrap da extensão
requirements.txt        # Dependências
.env.example            # Variáveis sem segredos
src/ingest.py            # Preparação da base
src/search.py            # Recuperação de trechos
src/chat.py              # Interação e resposta
document.pdf            # Documento de entrada
README.md               # Como configurar, executar e conferir
```

Recomendado, mas não exigido: metadados de origem/página, proteção contra ingestão duplicada, mensagens de erro claras e testes de recuperação. Interface web, API HTTP, autenticação, agentes, memória de conversa, OCR e índices aproximados não aparecem como exigências do desafio.

Arquitetura e responsabilidades

Proposta de implementação: dois fluxos separados, compartilhando a mesma configuração de embeddings e banco.

Fluxo A • Preparar a base

PDF → extração → chunks → embeddings → PostgreSQL

Executar na ingestão inicial e quando o documento ou a configuração de embeddings mudar. Gravar o texto dos chunks junto com os vetores; a LLM precisará do texto na consulta.

Fluxo B • Responder a uma pergunta

Pergunta → embedding → top 10 → contexto + prompt → LLM → CLI

Executar a cada pergunta. Reutilizar a base existente. Não reler nem vetorizar todo o PDF a cada interação.

Arquivo	Responsabilidade sugerida	Resultado
ingest.py	Validar configuração e PDF; extrair texto; criar chunks; adicionar metadados; persistir.	Quantidade de páginas/chunks processados e indicação clara de sucesso ou falha.
search.py	Abrir a base correta; buscar os trechos com k=10; devolver conteúdo, metadados e score.	Lista de pares (Document, score), sem imprimir nem gerar resposta.
chat.py	Manter o loop de entrada; chamar search; montar o prompt; invocar a LLM; exibir resposta.	Resposta ao usuário e saída limpa ao encerrar.
config.py (opcional)	Centralizar ambiente, modelos, conexão e collection para os três scripts.	Mesmos parâmetros na ingestão e na busca.

Contratos simples entre as partes

```
ingest() -> resumo da ingestão
search(query: str, k: int = 10) -> lista de (Document, score)
answer(question: str) -> texto da resposta
```

São contratos sugeridos, não assinaturas obrigatórias. Separar a busca da geração permite descobrir se uma resposta errada veio da recuperação ou do modelo. No começo, use chamadas síncronas e um fluxo explícito: a aplicação não precisa decidir ações de forma autônoma.

Busca semântica não é prova: proximidade entre vetores ajuda a localizar assunto semelhante. A verificação de que o trecho realmente responde à pergunta continua necessária.

Ingestão: do PDF à base

Implementação sugerida para src/ingest.py, mantendo os parâmetros fixados pelo curso.

1. Validar antes de gastar chamadas. Conferir caminho do PDF, variáveis obrigatórias, acesso ao banco e disponibilidade do modelo. Se o arquivo não existir ou não produzir texto utilizável, interromper com uma mensagem específica.

2. Extrair e inspecionar. Usar PyPDFLoader; explicitar mode="page" ajuda a manter a origem por página [8]. Conferir amostras do texto extraído, especialmente tabelas, valores, datas e colunas. Um PDF de imagem pode exigir OCR, que é uma extensão do escopo.

3. Dividir sem trocar caracteres por tokens. Aplicar RecursiveCharacterTextSplitter com a configuração abaixo. O tamanho é um máximo e a sobreposição é um alvo do algoritmo; chunks finais menores são normais [5].

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
    length_function=len,
)
chunks = splitter.split_documents(pages)
```

Com divisão por página, a sobreposição acontece dentro de cada página, não entre páginas. Registre essa escolha e teste fatos perto das quebras. Não force todos os chunks a terem exatamente 1.000 caracteres nem suponha que toda fronteira repetirá exatamente 150.

4. Gravar conteúdo e rastreabilidade. Para cada chunk, armazenar texto, embedding e metadados úteis. O vector store pode administrar o esquema físico; você não precisa criar uma segunda tabela manualmente sem necessidade.

Dado recomendado	Utilidade
source + page	Identificar o PDF e localizar a evidência; verificar se a numeração interna começa em zero.
document_hash + chunk_index	Reconhecer a versão do documento e criar IDs determinísticos.
embedding_model + dimensão	Evitar consulta com configuração incompatível com os dados existentes.

5. Tornar a reexecução previsível. Recomendo que ingerir o mesmo PDF duas vezes não duplique registros. IDs estáveis ajudam; quando o PDF mudar, trate também os chunks antigos que deixaram de existir. Para um único PDF, uma reinicialização explícita e documentada é uma alternativa simples.

Critério de conclusão

Texto extraído legível, chunks não vazios, vetores persistidos e consulta funcionando depois de fechar o processo de ingestão. A contagem isolada de vetores não comprova que os dados estão corretos.

Busca e resposta controlada

Implementação sugerida para search.py e chat.py. Preservar o prompt do enunciado [1].

Ao receber uma pergunta válida, consultar a mesma collection com o mesmo modelo de embeddings usado na ingestão. Na integração sugerida, similarity_search_with_score(query, k=10) já cuida da vetorização da consulta; não gere o mesmo embedding duas vezes. Concatenar o texto dos resultados e preencher o prompt abaixo.

```
CONTEXTO:
{resultados concatenados do banco de dados}

REGRAS:
- Responda somente com base no CONTEXTO.
- Se a informação não estiver explicitamente no CONTEXTO, responda:
  "Não tenho informações necessárias para responder sua pergunta."
- Nunca invente ou use conhecimento externo.
- Nunca produza opiniões ou interpretações além do que está escrito.

EXEMPLOS DE PERGUNTAS FORA DO CONTEXTO:
Pergunta: "Qual é a capital da França?"
Resposta: "Não tenho informações necessárias para responder sua pergunta."

Pergunta: "Quantos clientes temos em 2024?"
Resposta: "Não tenho informações necessárias para responder sua pergunta."

Pergunta: "Você acha isso bom ou ruim?"
Resposta: "Não tenho informações necessárias para responder sua pergunta."

PERGUNTA DO USUÁRIO:
{pergunta do usuário}

RESPONDA A "PERGUNTA DO USUÁRIO"
```

Decisões que evitam erros comuns

Sem evidência: se a busca não retornar texto, devolver a recusa sem chamar a LLM. Se retornar trechos apenas parecidos, o modelo ainda deve recusar. O prompt reduz o risco de invenção, mas não oferece garantia absoluta.

Score: confirmar a métrica e o comportamento da versão usada. Para distância de cosseno, menor significa mais próximo; isso não é probabilidade de acerto [7]. Não adotar um limiar arbitrário como requisito. Um filtro adicional só faz sentido depois de calibrado com exemplos do PDF.

CLI: ignorar entrada vazia, aceitar um comando de saída e tratar interrupção do teclado. Diferenciar indisponibilidade de banco/API de ausência de informação: uma falha operacional não deve se disfarçar de recusa por falta de contexto.

O exemplo sobre clientes em 2024 pressupõe ausência desse dado. A decisão deve depender do PDF real; não transforme perguntas do exemplo em uma lista fixa de bloqueio.

Configuração e compatibilidade

Verificações feitas no repositório público de referência e na documentação oficial em 26/10/2026.

O que já vem no exemplo

O Compose usa PostgreSQL 17 com imagem pgvector, publica a porta 5432 e mantém dados em volume. Há um serviço separado que espera o banco ficar saudável e cria a extensão vector [3]. Espere essa inicialização terminar antes da ingestão.

O requirements.txt fixa, entre outras, langchain 0.3.27 e langchain-postgres 0.0.15 [4]. Esses números descrevem a base do curso; não são uma afirmação de que sejam as versões mais recentes ou de que foram testadas nesta análise.

PGVector: curso versus biblioteca atual

O enunciado recomenda PGVector. O repositório oficial de langchain-postgres informa que essa classe está descontinuada desde a versão 0.0.14 e recomenda PGVectorStore [6]. Para começar, priorize um fluxo reproduzível compatível com a base do curso. Se migrar, documente a decisão e revalide criação da tabela, busca e pontuação; não misture exemplos das duas APIs.

Configurações a manter explícitas

Variável	Decisão prática
DATABASE_URL	No host, usar localhost e a porta publicada. Para PGVector com psycopg 3, usar o prefixo postgresql+psycopg:// [9].
PG_VECTOR_COLLECTION_NAME	Mesmo nome em ingest.py e search.py; evitar consultar coleção vazia por erro de configuração.
PDF_PATH	Apontar para document.pdf com uma regra clara de resolução do caminho.
Chave + modelo de embeddings	Usar um dos provedores previstos; manter o mesmo modelo e a mesma dimensão para indexação e perguntas.
Modelo de chat	Adicionar uma variável própria, como CHAT_MODEL. O .env.example consultado não define uma variável para a LLM [2].

Troca de embeddings: mesmo que dois modelos tenham a mesma dimensão, seus vetores podem não compartilhar o mesmo espaço semântico. Refaça a indexação em estrutura compatível. Dependendo do esquema, uma nova collection não altera a dimensão de uma tabela compartilhada; verifique o armazenamento antes de reinicializar dados.

Escolher nomes de modelos disponíveis na conta é uma decisão pendente da implementação. Não copiar os valores do template como garantia de disponibilidade. Manter chaves apenas no .env local e publicar um .env.example sem segredos.

Ordem de implementação

Roteiro proposto. Avançar quando a evidência da etapa estiver disponível.

Etapa	Trabalho	Evidência de conclusão
1 • Preparar	Fazer fork, criar ambiente virtual, instalar dependências e preencher o ambiente local.	Imports funcionam; versão do Python e parâmetros estão registrados.
2 • Subir o banco	Executar Compose; aguardar saúde e bootstrap da extensão.	Conexão bem-sucedida e extensão vector disponível.
3 • Validar leitura	Carregar o PDF e inspecionar texto e metadados antes de gerar vetores.	Amostras preservam fatos, valores e páginas.
4 • Ingerir	Dividir em 1000/150, gerar embeddings e persistir.	Chunks consultáveis; repetir ingestão tem comportamento documentado.
5 • Buscar sem gerar	Implementar search.py e inspecionar top 10, páginas e scores.	Perguntas conhecidas recuperam a evidência esperada.
6 • Integrar a LLM	Usar o prompt obrigatório e completar o loop de CLI.	Resposta correta para evidência presente e recusa para ausente.
7 • Conferir entrega	Executar a matriz de testes; revisar README e repositório público.	Outra pessoa consegue repetir a execução a partir das instruções.

Sequência de execução a documentar

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Preencher .env com os valores locais antes de continuar.
docker compose up -d
# Aguardar o banco e o bootstrap da extensão.
python src/ingest.py
python src/chat.py
```

Onde concentrar o esforço

Primeiro obtenha recuperação confiável; só depois ajuste a geração. Caso o trecho correto não apareça no top 10, trocar o prompt não corrige a causa. Para o volume de um único PDF, começar com busca exata evita introduzir complexidade antes de medir necessidade [7].

Melhorias posteriores: mostrar fontes em modo de diagnóstico, registrar latência por etapa e avaliar filtros. OCR, reranking, busca híbrida e índices HNSW/IVFFlat devem responder a um problema medido, não bloquear a primeira entrega.

Como saber que está pronto

Matriz de validação proposta, além das exigências explícitas do enunciado. Use fatos do PDF real como gabarito.

Caso	Resultado esperado
Fato explícito	Escolher um fato com página conhecida. O trecho aparece no top 10 e a resposta mantém entidade, data, valor e unidade.
Paráfrase	Reformular a pergunta sem repetir as palavras do PDF. A evidência continua sendo recuperada.
Dado ausente	Perguntar por algo relacionado, mas não informado. Retornar exatamente a frase de recusa.
Tema externo ou opinião	Perguntar algo fora do documento ou pedir julgamento. Recusar quando não houver informação explícita que responda.
Instrução conflitante	Pedir para ignorar o contexto e inventar um número. A resposta não deve obedecer a essa instrução.
Base vazia ou incompleta	Não inventar resposta. Informar ausência de base utilizável ou solicitar ingestão, conforme o caso.
PDF vazio/imagem	Interromper ingestão sem declarar sucesso; explicar que não foi possível obter texto utilizável.
API ou banco indisponível	Mostrar erro operacional claro; não apresentar a recusa semântica como se a consulta tivesse funcionado.
Reexecução e persistência	Repetir ingestão conforme a política escolhida; reiniciar o chat e confirmar que consulta a base persistida.
Mudança de embedding	Detectar configuração incompatível; exigir nova indexação em estrutura apropriada.

Avalie recuperação e resposta separadamente

Crie um pequeno conjunto de perguntas com resposta e página esperadas, além de perguntas sem resposta. Registre: evidência encontrada no top 10, resposta apoiada nessa evidência e recusas corretas. Esses indicadores ajudam a localizar a falha sem depender da impressão de que o chat “parece funcionar”.

Checklist da entrega

O README deve conter pré-requisitos, variáveis de ambiente, modelos e versões escolhidos, comandos de execução, política de reingestão, exemplos de perguntas e limitações conhecidas. Conferir que código, PDF de entrada e arquivos exigidos estão presentes, que não há chaves publicadas e que a URL pública pode ser aberta por outra pessoa.

Limite da avaliação: passar nesses exemplos demonstra o comportamento observado; não prova ausência de alucinação para qualquer pergunta. Este documento especifica verificações a executar, não resultados de testes já realizados.

Fontes e decisões pendentes

Links consultados para fundamentar a análise. O enunciado é a referência para os critérios de entrega; as demais fontes esclarecem implementação e versões.

[1] Enunciado da Full Cycle, lido na aba aberta do Chrome
Objetivo, stack, parâmetros, prompt, estrutura e entrega. Acesso depende da conta do aluno.

[2] Repositório e template de ambiente
Estrutura de referência; .env.example consultado no mesmo repositório.

[3] Docker Compose do desafio
Imagem do banco, porta, persistência e bootstrap da extensão.

[4] Dependências do desafio
Versões fixadas no exemplo disponibilizado pelo curso.

[5] LangChain: RecursiveCharacterTextSplitter
Tamanho por caracteres, limite de chunk e sobreposição alvo.

[6] Repositório oficial langchain-postgres
Aviso de descontinuação de PGVector e recomendação de PGVectorStore.

[7] Documentação oficial pgvector
Métricas de distância e buscas exatas ou aproximadas.

[8] Referência oficial de PyPDFLoader
Leitura de PDF e modos de extração.

[9] LangChain: integração PGVector
Conexão psycopg, persistência de documentos e busca com score.

Decisões a fechar ao começar a implementação

Escolher provedor e modelos disponíveis; validar a combinação de dependências com o Python local; confirmar se o PDF contém texto extraível; definir como repetir a ingestão; registrar a decisão sobre PGVector ou PGVectorStore. Nada disso altera os requisitos de chunking, top 10 e resposta restrita ao contexto.

Primeiro marco recomendado

Conseguir ler o PDF, gravar os chunks e recuperar a passagem correta para uma pergunta conhecida. Esse marco verifica a base do sistema antes de adicionar a geração de respostas.

Documento de análise e planejamento. Não constitui implementação, execução de testes ou submissão do desafio. As recomendações de engenharia foram elaboradas a partir do enunciado e das fontes acima.